Online Payments

Easypaisa – Pinless MA Precheck API

Payment Method – Integration Document

Table of Contents

1. Easypay Payment Solution 3
2. Pre Check API 3
3. Integration Parameters 4
   3.1 Pre-Check API 4
      3.1.1 Request Parameters 4
      3.1.2 Response Parameters 4
      3.1.3 Sample Request 5
      3.1.4 Sample Response 5
   3.2 Response Codes for Pre-check API 5
4. RSA Encryption 7
   4.1 Public/Private Key Exchange 7
   4.2 Signature Generation 8
   4.3 Signature Verification 9

1. Easypay Payment Solution

Easypay is an electronic payment solution that easily and seamlessly integrates with any online website and/or shopping cart and enables users to pay online through Payment Gateway. Easypay is built upon the latest state of the art technologies and toolset that contains all features a secure payment solution system must have. The users require no special technology or business license; only a valid email address to have an account on Easypay.

2. Pre Check API

Easypay Pinless MA APIs enables online merchants/webstores to allow their customers to pay from their Easypaisa Mobile Accounts (MA), but without being prompted for the PIN. Merchant applications can use these APIs to generate a token for Easypaisa customers and then use these tokens to create payment transactions.Transaction Flow

3. Integration Parameters

The request header for all APIs in this section will be as follows:

3.1 Pre-Check API

This API is used to perform pinless MA transactions

URL: *https://easypay.easypaisa.com.pk/easypay-service/rest/pinless/v1.0/daraz-prechecks*

3.1.1 Request Parameters

Parameter	Explanation	Condition	Data Type
Request			
token	Pinless token for the linked account	Mandatory	String
orderId	Order id for which pre-check is called	Mandatory	String
storeId	Store ID through which order is being placed	Mandatory	String
mobileAccountNo	According to the new validation rules, the Mobile Numbers passed must be 11 digits in length and must start with 03. There should be no alphabets or special characters in the number.	Mandatory	String
Amount	Order amount	Mandatory	String
Signature			
signature	RSA signature (see section 4 for instructions to generate)		String

3.1.2 Response Parameters

Parameter	Explanation	Condition	Data Type
Response			
errorCode	Error Code identifying transaction result.		String
errorDescription	Reason describing the Error code. (see section **Error! Reference source not found.** for possible response codes and descriptions)		String
Signature			
signature	RSA signature (see section 4 for instructions to verify)	Mandatory	String

3.1.3 Sample Request


```
{
 "request":
        {
"token": "XCGNhuy",
“orderId”: ABC-01
"storeId": "641",
                "mobileAccountNo": "03453101535"
"amount": "45.00"

        },
 "signature":
"as8wYWy/Dc/p67iJyIOHoq9S92GOSW2UAblQl4YCLIcjYrKD+IeyVO7p61a71UCfuMuk0+aSDpKocliSjsHBrl2yN1KEW2UlU
WlRDdd7W2+U0CVtBkK7jjk27CR+TpM6Bq/9+40EXVEzi5C0zwWf6LHHpW1cxdPAHj0gw8BhwwSv5UvW7HMnb9M27ffi
AR2iIFOOiQ1jj+HVyNfb+JD5YP38vZyZmGC4UshXhxVp/UEstZssg3Tc0FHsxq8CWz2CV6fBOqsOyRGTXWIYGBxfgxPDpYlqr
n/FIOzGGwBABi9AvKHARn0qZb6LLn1wqaQQt76R0gvR795xiZvkrU98tA=="
```


3.1.4 Sample Response


```
{
 "response":
        {
                "errorCode": "003",
                "errorDescription": "Insufficient Balance"
        },
"signature":
"as8wYWy/Dc/p67iJyIOHoq9S92GOSW2UAblQl4YCLIcjYrKD+IeyVO7p61a71UCfuMuk0+aSDpKocliSjsHBrl2yN1KEW2UlU
WlRDdd7W2+U0CVtBkK7jjk27CR+TpM6Bq/9+40EXVEzi5C0zwWf6LHHpW1cxdPAHj0gw8BhwwSv5UvW7HMnb9M27ffi
AR2iIFOOiQ1jj+HVyNfb+JD5YP38vZyZmGC4UshXhxVp/UEstZssg3Tc0FHsxq8CWz2CV6fBOqsOyRGTXWIYGBxfgxPDpYlqr
n/FIOzGGwBABi9AvKHARn0qZb6LLn1wqaQQt76R0gvR795xiZvkrU98tA=="
}
```


3.2 Response Codes for Pre-check API

Response Code	Response Description
0000	Success
0001	System Error
0002	Required Field Missing
0003	Invalid Mobile Number
0004	Invalid Store
0005	Invalid Amount
0006	Invalid Token
0007	Invalid Signature

Response Code	Response Description
0008	Bad Credentials
0009	Account does not exist
0010	Account is inactive
0011	Store Not Active
0012	Pinless Not Enabled
0013	Expired Token
0014	Insufficient Balance
0015	Key Not Uploaded

4. RSA Encryption

In order to use these Payment Services, it is necessary for the client to send a digital signature along with each API request. This signature is used by Easypaisa to authenticate the client before processing their request. The steps required to perform this encryption are described below:

Client and Easypaisa must exchange RSA keys before making API calls, and the length of RSA key must be 2048 bits. When making API call to Easypaisa, client uses its RSA private key to sign the API request. After receiving the API request, Easypaisa will use the client's RSA public key to verify whether the signature matches the content of API request. Similarly, when client receives the API response, it is highly recommended that client verifies the signature of API response by using Easypay's RSA public key.

4.1 Public/Private Key Exchange

An RSA key pair consists of a private key and a public key. The private key is required for generating the signature, while the public key is used for verifying the signature. Many tools can be used to generate the RSA key pair. After the RSA2048 key pair is generated, you must exchange the public key with the Easypaisa server for signature verification. The following steps must be used for this purpose:

1. **Upload your public key in Easypaisa merchant portal.**

Log in to the portal and visit the following path: "*Account Settings -> Public Key Configuration -> Upload Public Key*". Upload your public key here.

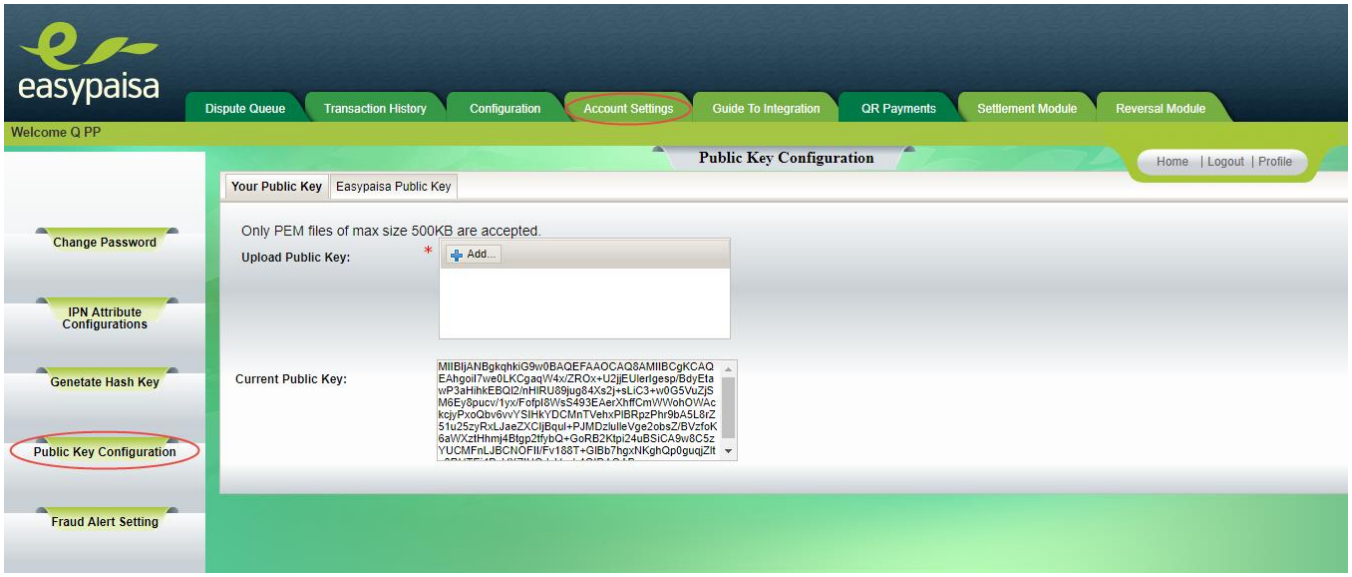

Figure 1 – Public Key Configuration (Upload)

2. **Download Easypaisa public key**. [OPTIONAL]

On the same path, enter the “Easypaisa Public Key” tab and click on the download button to download Easypaisa public key.

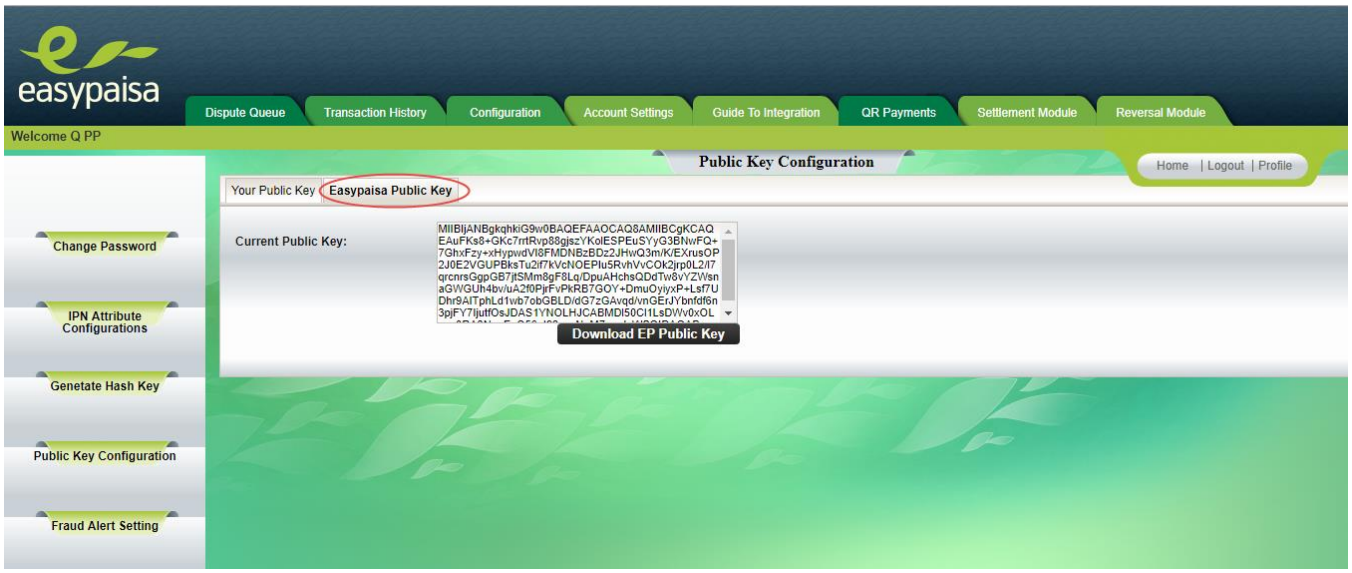

Figure 2 – Public Key Configuration (Download)

4.2 Signature Generation

The following figure illustrates a sample API request. The content enclosed by the two curly braces (inclusive) is what needs to be signed to create the RSA signature. And the generated signature is put in the *signature* parameter.

Figure 3 - Sample Request

1. Extract the content to be signed. For example:

```
{"storeId":"123","transactionAmount":"100","orderId":"MSTest1000","cardNumber":"5313581000123430","expiryDate":"0521","cvv":"100","mobileNumber":"03451234567","email":"imran.khan@easypay.com","postBackUrl":"http:merchantwebsite.com/postBackUrl.php"}
```

2. Hash the JSON content by using the SHA256 withRSA algorithm. Then use the client's RSA private key to sign the value to obtain the signature. To achieve a better security level, the length of the RSA key/pairs must be 2048 bits. The following sample illustrates a raw signature:


```
sXSGdA4tR4B0wnF6Nr8ShpfNMtHISWuzLpdKoNWskIgCcspMVFFzgkez4BrmtYXkMqZh1tyz/8sNNU3F2VTu0fO
x+ZP4CnmVjNu8bciu4hTtnytA3Y1GZ/yXAUD+mVYGXeyDO3fHrqDdI7k3xV/Q8FCOd0culTsN6RRMxMZq+oqR2x
+EjOXVofp7BZJwhQE8Us4Al573FQz5EHDvCLx35lq2hoMhXLCVLVDBHcpCgotuY4mg91MXh/qqcFGY/hQKhGdWA
bZ9tcp2Q5s2ESh37rqxnOwZrrepK8NC6vMBPUiSPxQefpev2nG0JzAHOjRPbBXYw1XBS6RBYKAmufZZiQ==
```


3. Base64-encode the signature. A sample result is listed below:


```
c1hTR2RBNHRSNEIwd25GNk5yOFNocGZOTXRISVNXdXpMcGRLb05Xc2tJZ0Njc3BNVkZGemdrZXo0QnJtdFlYa01
xWmgxdHl6LzhzTk5VM0YyVlR1MGZPeCtaUDRDbm1Wak51OGJjaXU0aFR0bnl0QTNZMUdaL3lYQVVEK21WWUdYZX
lETzNmSHJxRGRJN2szeFYvUThGQ09kMGN1bFRzTjZSUk14TVpxK29xUjJ4K0VqT1hWb2ZwN0JaSndoUUU4VXM0Q
Ww1NzNGUXo1RUhEdkNMeDM1bHEyaG9NaFhMQ1ZMVkRCSGNwQ2dvdHVZNG1nOTFNWGgvcXFjRkdZL2hRS2hHZFdB
Ylo5dGNwMlE1czJFU2gzN3JxeG5Pd1pycmVwSzhOQzZ2TUJQVWlTUHhRZWZwZXYybkcwSnpBSE9qUlBiQlhZdzF
YQlM2UkJZS0FtdWZaWmlRPT0=
```


4. Use the obtained string as the value of the Signature parameter.

4.3 Signature Verification

A sample response is shown as below:


```
{
    "response": {
        "responseCode": "0000",
        "responseDesc": "SUCCESS",
        "storeId": "2200",
        "storeName": "XXX",
        "orderId": "qqq558",
        "transactionAmount": "13.00",
        "gatewayCode": "NOT_ENROLLED_NO_ERROR_DETAILS",
        "transactionStatus": "PENDING",
        "transactionDateTime": "Thu Jan 03 13:13:43 UZT 2019"
    },
    "signature": "rdPwPlZ561mXk/iOvIJN3p8cb7ivXDaaAhZOLyQvBOfOw8s+h6EzTOjmABLQ8+swtdBzACHuZ3q8do+cjeetqaQL4eSdXc0sEpdzP96H+Cs6
        /fizztSpN83eQf0EI8saFIs+dKBqQuBlEp4740KI1In/RFPCgVBTGi5NnxlQyI7bN0r0j8kxgrqYcFDxpA5odWbZjHFkYO6pqFQAhM7RoF6gE4W108Pc3FwxJCs1bJKkpN
        /CHqaPld0asgLAc+8alh0rx5Ji15Aj3r+ZX8LtNOenyGSkxGrZkk7VLnH8zvM4ZhOHF7S6NXHUQJWVvak634QpUk4WD41A+L3kZz06xw=="
}
```


The enclosed content (inclusive) is what needs to be verified against the signature using Easypay's RSA Public Key

After receiving a response, perform the following steps to verify the signature:

1. Split the full response contents to 2 parts, the response JSON string and the signature string, by using the regular expression instead of JSON object.
2. Hash the response JSON string by using the SHA256 algorithm to obtain a message digest.
3. Use the public key to decrypt the signature to a message digest.
4. Compare the two message digests obtained in step 2 and step 3. If the digests are the same, then it indicates that the signed data has not been changed.